自然语言处理课程实验报告

课程： 自然语言处理
作业： 作业一 · 文本分类实现
数据集： NYT 新闻数据集、AG News
代码仓库： https://github.com/RowinNyan/NKU-NPL-HW1

一、实验目的

本实验围绕文本分类任务，系统比较三类文本表示方法在同一数据划分下的实际效果：

1. 理解词袋模型、词嵌入（Word Embedding）与预训练语言模型三种文本表示的基本思想；
2. 掌握文本分类任务的基本实验流程，包括数据划分、分词、向量化、模型训练与评价；
3. 熟悉 Logistic Regression 在文本分类中的使用；
4. 掌握 Word2Vec、GloVe 等词向量的训练与使用方法；
5. 熟悉 BERT 类预训练语言模型的微调流程；
6. 能够使用 Accuracy 与 Macro-F1 对文本分类模型进行评价，并理解两个指标在不同类别分布下的差异；
7. 能够对不同文本表示方法的性能进行比较与分析，解释性能差异的来源。

二、实验环境

项目	配置
操作系统	Windows（Git 2.43.0）
CPU	12 逻辑核心
内存	15.7 GB
GPU	NVIDIA GeForce RTX 4060 Laptop，8 GB 显存，驱动 572.83
Python	3.10.11（项目内独立虚拟环境 `.venv`）
深度学习框架	PyTorch 2.6.0+cu124（CUDA 12.4 运行时），bf16 混合精度
预训练模型库	transformers 5.18.0，accelerate 1.15.0

项目	配置
机器学习库	scikit-learn 1.7.2，gensim 4.4.0
数据处理	pandas 2.3.3，numpy 2.2.6，scipy 1.15.3
分词	nltk 3.10.3（`word_tokenize`，punkt / punkt_tab）

三、数据集

3.1 NYT 新闻数据集

`datasets/nyt.csv`，两列 `text` / `label`，无缺失值。

项目	数值
样本数	11519
类别数	3
类别分布	business 1429（12.41%）、politics 1451（12.60%）、sports 8639（75.00%）
文档长度（nltk token）	中位数 768，均值 730，P95 1174，最大 5977

数据集类别分布高度不均衡，sports 占 75%，这是本次实验必须同时报告 Macro-F1 的直接原因（详见 4.2 节）。

3.2 AG News 数据集

`datasets/ag.csv`，仅 `text` 一列，无标签。

项目	数值
样本数	90000
文档长度（按空白切分）	中位数 39，均值 39.2，P95 53，最大 152

按作业要求，AG News 在本次实验中不用于分类训练，仅用于训练 Word2Vec 词向量。

四、统一实验设置

为保证不同文本表示方法之间严格可比，以下设置在三个 Task 中完全一致。

4.1 数据划分

- 对 11519 条样本用 `numpy.random.default_rng(42)` 随机打乱，按 80% / 10% / 10% 切分。
- 实际规模：train / val / test = 9215 / 1151 / 1153。
- 划分结果以行索引形式缓存于 `splits/nyt_split_seed42_n11519.pkl`。使用索引而非行序，意味着即使原始 csv 行序变化，划分也完全一致；三个 Task 均从该文件读取同一份划分。
- 最终性能统一在 NYT Test Set（n=1153）上评价，验证集用于观察训练过程。

4.2 评价指标

- Accuracy：分类正确的样本数占全部测试样本的比例。
- Macro-F1：先分别计算每个类别的 F1，再对类别取算术平均（各类别等权）。

本数据集 sports 占 75%，Accuracy 会被多数类主导，而 Macro-F1 对少数类更敏感，因此两个指标需结合解读。

所有指标的计算口径统一由 `common/modeling.py` 中的 `evaluate_predictions()` 实现，三个 Task 共用同一函数，避免因实现差异引入偏差。

4.3 分词与词表

- Task 1 / Task 2：使用 `nltk.word_tokenize`，统一转小写（与 GloVe-6B、Word2Vec 的小写词表对齐），不做去停用词、词干化或标点过滤。
- Task 3：BERT 使用其自带的 WordPiece 分词器，与 nltk 分词无关。
- 词表（Task 1）：仅由训练集构建，以避免验证集/测试集信息泄漏；词表按字典序编号，使特征下标与语料中词的出现顺序无关，保证可复现。词表大小 |V| = 127352。

4.4 分类器

Task 1 与 Task 2 统一使用 `sklearn.linear_model.LogisticRegression`，超参数 `C=1.0`、`max_iter=1000`，未做额外调参，以保证两组实验的差异只来自文本表示方式。

五、Task 1：Bag of Words + Logistic Regression（30 分）

5.1 方法

两种词袋表示，均为 `|V|` 维稀疏向量（|V| = 127352）：

1. Binary Bag of Words：`x_i = 1` 若词 `i` 出现在文档中，否则为 0。只考虑词是否出现，不考虑出现次数。
2. Word Frequency：`x_i` 为词 `i` 在文档中的出现次数。

向量化时词表外的词（OOV）直接忽略。由于 9215 × 127352 的稠密矩阵规模约 1.2×10^9 个元素，实现上采用 CSR 稀疏矩阵存储，实际非零占比仅 0.25%。

5.2 实验结果

Test Set（n=1153）

表示方法	Accuracy	Macro-F1	LR 训练用时	迭代次数
Binary BoW	0.9870	0.9717	1.98 s	18
Word Frequency	0.9896	0.9751	15.53 s	141

Validation Set（n=1151）

表示方法	Accuracy	Macro-F1
Binary BoW	0.9800	0.9579
Word Frequency	0.9852	0.9676

分类别指标（Test Set）

Binary BoW：

类别	Precision	Recall	F1	Support
business	0.9769	0.9270	0.9513	137

类别	Precision	Recall	F1	Support
politics	0.9494	0.9868	0.9677	152
sports	0.9954	0.9965	0.9960	864

Word Frequency：

类别	Precision	Recall	F1	Support
business	0.9847	0.9416	0.9627	137
politics	0.9490	0.9803	0.9644	152
sports	0.9977	0.9988	0.9983	864

混淆矩阵（行 = 真实，列 = 预测）

Binary BoW：

	business	politics	sports
business	127	7	3
politics	1	150	1
sports	2	1	861

Word Frequency：

	business	politics	sports
business	129	7	1
politics	2	149	1
sports	0	1	863

5.3 结果分析

1. 词频略优于二值，但优势很小。Word Frequency 相比 Binary BoW 在测试集上 Accuracy

提升 0.26 个百分点、Macro-F1 提升 0.34 个百分点。词频保留了词的强度信息，对判别性关键词（sports 新闻中反复出现的队名、赛事词）有额外区分度；但提升有限，说明在这个数据集上"词是否出现"已经提供了绝大部分判别信息。代价是训练迭代次数从 18 次增至 141 次（特征取值尺度更大、条件数更差），LR 训练用时约为二值的 7.8 倍。

2. Accuracy 明显高于 Macro-F1（约 1.5 个百分点）。原因是类别严重不均衡：sports 占 75.0%（support 864/1153）且其 F1 高达 0.996，而样本稀少的 business / politics 只有 0.95 ~ 0.97。Accuracy 被多数类主导，Macro-F1 对每个类别等权，因而更能反映模型在少数类上的真实能力。这正是作业要求同时报告两个指标的意义。
3. 错误高度集中在 business ↔ politics。两种表示的共 15 / 12 个测试错误中，有 8 ~ 9 个发生在 business 与 politics 之间（business 被误判为 politics 各 7 次，反向 1 ~ 2 次），而 sports 几乎不与另两类混淆。business 的召回率最低（0.927 / 0.942），politics 的精确率最低（均为 0.949）——这与两类新闻主题天然重叠（政治新闻常涉及经济、经济新闻常涉及政策）相符。

六、Task 2：Word Embedding / Word2Vec（50 分）

6.1 方法

三种 100 维词向量，分类器仍为 Logistic Regression（C=1.0，max_iter=1000）：

1. Pre-trained GloVe：使用 `glove.6B.100d.txt`（400000 词 × 100 维），从 `glove.6B.zip` 解压得到。
2. Word2Vec trained on AG News：在 AG News 的 90000 条文本上自训 100 维 Word2Vec。
3. Word2Vec trained on NYT：在 NYT 文本上自训 100 维 Word2Vec。

文档向量按作业要求取文档内所有有效单词（在词向量表中查得到的词）词向量的平均：

$$\mathbf{e}(d) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{e}(w_i)$$

查不到的 OOV 词直接跳过；若一篇文档一个词都查不到，则文档向量为零向量。

6.2 Word2Vec 超参数

作业未规定 Word2Vec 的具体超参数，本实验采用 Word2Vec 原论文与 gensim 的常用默认设置，两组自训模型完全一致：

参数	取值
`vector_size`	100
`window`	5
`min_count`	5
`sg`	0（CBOW）
`negative`	5
`epochs`	5
`workers`	1（保证结果可复现）
`seed`	42

6.3 关于 NYT 自训语料范围的说明

作业原文为"使用 NYT 文本数据训练 Word2Vec"，未明确是否包含验证/测试集文本。本实验的做法是：

- 主结果采用无泄漏设置：Word2Vec 仅用训练集（9215 篇）训练，记为 `NYT train split`；
- 附加对照：用全部 NYT（11519 篇）训练，记为 `NYT full`。该设置下词向量在训练时"见过"测试集文本，属于 transductive 设置，不作为主结果，仅用于观察同域语料规模带来的影响。

6.4 实验结果

Test Set（n=1153）

表示方法	词表大小	token 覆盖率	Accuracy	Macro-F1	LR 训练用时
GloVe-6B-100d (pre-trained)	400000	98.07%	0.9818	0.9585	0.38 s
Word2Vec-100d (AG News)	25376	94.25%	0.9809	0.9553	0.32 s
Word2Vec-100d (NYT train split)	31574	97.87%	0.9853	0.9633	0.24 s
Word2Vec-100d (NYT full)	35447	98.15%	0.9861	0.9676	0.23 s

> 注：表中"LR 训练用时"仅为分类器拟合时间。文档向量为 11519 × 100 的稠密矩阵，向量化本身耗时约 4~6 秒。

Validation Set（n=1151）

表示方法	Accuracy	Macro-F1
GloVe-6B-100d (pre-trained)	0.9748	0.9465
Word2Vec-100d (AG News)	0.9713	0.9418
Word2Vec-100d (NYT train split)	0.9731	0.9435
Word2Vec-100d (NYT full)	0.9791	0.9583

分类别 F1（Test Set）

表示方法	business	politics	sports
GloVe-6B-100d	0.9254	0.9548	0.9954
Word2Vec (AG News)	0.9219	0.9481	0.9960
Word2Vec (NYT train split)	0.9398	0.9518	0.9983
Word2Vec (NYT full)	0.9478	0.9579	0.9971

混淆矩阵（Test Set，行 = 真实，列 = 预测）

GloVe-6B-100d：

	business	**politics**	**sports**
business	124	9	4
politics	4	148	0
sports	3	1	860

Word2Vec (AG News)：

	business	**politics**	**sports**
business	124	9	4
politics	6	146	0
sports	2	1	861

Word2Vec (NYT train split)：

	business	**politics**	**sports**
business	125	10	2
politics	4	148	0
sports	0	1	863

Word2Vec (NYT full)：

	business	**politics**	**sports**
business	127	8	2
politics	3	148	1
sports	1	1	862

6.5 结果分析

1. 词向量平均整体不如词袋。四组词向量实验的 Macro-F1 落在 0.955 ~ 0.968，而 Task 1 的 Word Frequency 为 0.9751、Binary BoW 为 0.9717。原因有两层：
   - 平均操作稀释了判别性强的词。NYT 文档平均 731 个 nltk token，一篇 sports 新闻里几十个队名、赛事词的信息，被大量中性词平均掉；
   - `min_count=5` 截断了词表。词袋保留了 127352 个词（含大量低频但高判别性的词），而 Word2Vec 词表只有 2.5 万 ~ 3.5 万。

   这说明在训练样本充足的主题分类任务上，稀疏高维词袋仍是很难超越的强基线。
2. 同域训练优于跨域训练。NYT 自训（0.9633）> 预训练 GloVe（0.9585）> AG News 自训（0.9553）。NYT 语料只有约 673 万 token，远少于 GloVe 的 60 亿 token，但同域语料学到的词向量更贴合新闻主题词，因而反超预训练词向量。领域匹配度比语料规模更重要。
3. AG News 模型与 GloVe 几乎打平，但覆盖率最低。AG News 词表仅 25376 词，对 NYT 的 token 覆盖率 94.25%，明显低于 GloVe 的 98.07%——这是跨域的直接体现（AG News 是短标题式新闻，NYT 是长篇正文，词汇更丰富）。覆盖率低 3.8 个百分点，Accuracy 却只差 0.09 个百分点：被丢掉的 OOV 多是低频词，对文档平均向量的影响本就很小。
4. `NYT full` 略优于 `NYT train split`，但优势不完全公平。Macro-F1 从 0.9633 提升到 0.9676（+0.43 个百分点），因为它见过测试集文本。提升幅度有限，说明同域词向量的主要收益来自领域匹配而非额外语料量。
5. business 依旧是最难的类别。四组词向量的 business F1 在 0.92 ~ 0.95，均低于词袋的 0.9627，与 Task 1 的结论一致：business 与 politics 主题重叠，而"一袋词求均值"的做法无法刻画主题边界。

七、Task 3：Pre-trained Neural Models（20 分）

7.1 方法

微调 `google-bert/bert-base-uncased`（109.5M 参数）做三分类。按作业规定设置：

- Maximum sequence length = 64
- Number of epochs = 3

作业未规定的超参数采用 BERT 微调的常规设置：

参数	取值
优化器	AdamW
学习率	2e-5
权重衰减	0.01
学习率调度	linear，warmup 10%（86 / 864 步）
Batch size	32（训练） / 64（评估）
精度	bf16 混合精度
随机种子	42

数据划分与指标口径同 Task 1/2（同一份 `splits/nyt_split_seed42_n11519.pkl`、同一个 `evaluate_predictions()`）。由于几乎所有文档都超过 64 个 token，序列长度恒为 64，无需动态 padding。

7.2 关键约束：max_len=64 造成的截断

用真实 WordPiece 分词器对全部 11519 篇文档做不截断编码，测得：

指标	数值
WordPiece 长度 中位数 / 均值 / P95 / 最大	873 / 835 / 1329 / 6775
能完整放进 64 个 token 的文档	5 / 11519（0.04%）
平均能看到的正文比例	10.6%
中位文档能看到的正文比例	7.3%

也就是说，每篇新闻平均有 89.4% 的正文被截掉，模型只读到开头约 49 个词。这是解读 Task 3 结果时最重要的前提。

7.3 训练过程

Epoch	val Loss	val Accuracy	val Macro-F1
1	0.0864	0.9670	0.9362
2	0.1090	0.9705	0.9426
3	0.0878	0.9757	0.9529

三轮指标稳定提升，验证 loss 在第 2 轮略升后回落，没有出现明显过拟合。GPU 上 3 个 epoch 共 864 步，训练总用时 98.93 秒。

7.4 实验结果

Test Set（n=1153）

指标	数值
Accuracy	0.9792
Macro-F1	0.9585

分类别指标（Test Set）

类别	Precision	Recall	F1	Support
business	0.9286	0.9489	0.9386	137
politics	0.9299	0.9605	0.9450	152
sports	0.9965	0.9873	0.9919	864

混淆矩阵（行 = 真实，列 = 预测）

	business	politics	sports
business	130	5	2
politics	5	146	1

	business	politics	sports
sports	5	6	853

7.5 结果分析

1. BERT 没能超过词袋。109.5M 参数的预训练模型取得 0.9585 的 Macro-F1，低于 Word Frequency 的 0.9751（差 1.66 个百分点），也低于 Binary BoW 的 0.9717。考虑到截断现象，这一结果并不意外。
2. 但 BERT 在只看到 10.6% 正文的条件下仍达到 0.9585，说明导语信息量很高。NYT 沿用路透社体例，首段导语通常点明"谁、在哪、发生了什么"，主题信息高度浓缩（例如 `(reuters) - carlos tevez sealed his move to juventus on wednesday...`）。预训练模型的语言理解能力使其能从这几十个词中提取出接近全文词袋的判别信息。
3. BERT 的 sports 召回率是全场最低（0.9873，其余方法均在 0.995 以上）。混淆矩阵显示 sports 被误判为 business 5 次、politics 6 次，共 11 次，明显多于其他方法。合理推断是：只读开头 64 个 token 时，部分体育新闻的导语（涉及球队交易、合同、联盟政策）在语义上更接近商业或政治新闻。这从反方向印证了截断确实造成了信息损失。
4. 训练过程没有过拟合，瓶颈在输入而非模型容量。验证指标持续上升、验证 loss 无显著反弹，说明 3 个 epoch 内模型尚未充分利用其容量。因此性能瓶颈更可能来自输入被截断，而非模型拟合能力不足。

八、三种文本表示方法的横向对比

8.1 总表（NYT Test Set，n=1153，按 Macro-F1 降序）

排名	表示方法	Task	Accuracy	Macro-F1	business F1	politics F1	sports F1
1	Word Frequency	1	0.9896	0.9751	0.9627	0.9644	0.9983
2	Binary BoW	1	0.9870	0.9717	0.9513	0.9677	0.9960
3	Word2Vec-100d (NYT full)	2	0.9861	0.9676	0.9478	0.9579	0.9971

排名	表示方法	Task	Accuracy	Macro-F1	business F1	politics F1	sports F1
4	Word2Vec-100d (NYT train split)	2	0.9853	0.9633	0.9398	0.9518	0.9983
5	GloVe-6B-100d	2	0.9818	0.9585	0.9254	0.9548	0.9954
6	BERT-base-uncased (max_len=64)	3	0.9792	0.9585	0.9386	0.9450	0.9919
7	Word2Vec-100d (AG News)	2	0.9809	0.9553	0.9219	0.9481	0.9960

8.2 对比分析

1. 表示能力呈清晰梯度：稀疏词袋 > 同域词嵌入 > 跨域词嵌入 > 被截断的预训练模型。
   最优方法（词频词袋，0.9751）与最差方法（AG News 自训词向量，0.9553）相差约 2 个百分点。值得注意的是，参数量与性能并不正相关：词袋的"参数"是 127352 维线性权重，BERT 有 109.5M 参数，但前者胜出。
2. 在这个数据集上，性能主要由两个因素决定，而非模型复杂度：
   - 信息覆盖量：词袋使用了全文所有词；BERT 只使用了 10.6% 的正文；词嵌入虽也基于全文，但被压缩成 100 维定长向量。
   - 领域匹配度：同为 100 维词嵌入，NYT 自训（673 万 token）显著优于 GloVe（60 亿 token），说明在领域内语料上训练比在大规模通用语料上预训练更有效。
3. 三个 Task 的瓶颈完全一致：business 与 politics 的混淆。
   无论何种表示，sports 的 F1 都在 0.99 以上，而 business 与 politics 始终是错误的主要来源。business 的 F1 随表示方式变化明显：词袋 0.9627 → NYT 词向量 0.9398 → BERT 0.9386 → GloVe 0.9254 → AG News 词向量 0.9219。越是丢失细节的表示，越难区分这对近邻类别，这与两类新闻主题天然重叠的事实相符。
4. Accuracy 与 Macro-F1 的差距随表示方式变化。
   所有方法的 Accuracy 都比 Macro-F1 高 1.5 ~ 2.6 个百分点，这是类别不均衡的必然结果。差距在词袋上最小（约 1.5 个百分点），在 AG News 词向量上最大（约 2.6 个百分点）——说明后者对少数类的区分能力更弱。

九、结论

1. 在本题的 NYT 三分类任务上，高维稀疏词袋 + Logistic Regression 取得了最佳性能（Macro-F1 0.9751）。词频表示略优于二值表示。这一结果说明，当训练样本量充足（9215 条）且类别可由关键词区分时，简单的词袋模型仍是极具竞争力的强基线。
2. 词嵌入的有效性主要取决于领域匹配度，而非预训练语料规模。仅用 673 万 token 的 NYT 自训 Word2Vec 超过了基于 60 亿 token 训练的 GloVe-6B，也超过了跨域的 AG News 自训词向量。文档向量取平均会稀释判别性词，`min_count` 截断会丢弃低频高判别性词，这两点是词嵌入在本任务上落后于词袋的主要原因。
3. BERT 的性能受 max_len=64 严重制约。该设置使模型平均只能看到 10.6% 的正文（11519 篇中仅 5 篇可完整放入），因此在只读导语的条件下取得 0.9585 的 Macro-F1 已属不易，但仍低于词袋。训练曲线显示模型并未过拟合，瓶颈在输入信息的截断而非模型容量。
4. 三个 Task 的错误结构高度一致，都集中在 business 与 politics 之间，说明这是数据集层面的固有难点（两类主题重叠），而非某种表示方法特有的缺陷。

十、不足与改进方向

1. 未做超参数调优。Logistic Regression 固定 C=1.0，Word2Vec 使用默认超参数，BERT 使用常规微调设置。作业未要求调参，但调参后各组方法的相对排序可能变化。
2. 未做类别不均衡处理。数据集 sports 占 75%，实验中未使用类别权重、重采样或 Focal Loss。从 Macro-F1 与 Accuracy 的差距看，针对少数类的处理可能带来提升。
3. 未验证放宽 max_len 的效果。本次严格按作业要求固定 max_len=64。基于截断分析可以预期，将 max_len 提升到 256 / 512（覆盖 30% ~ 60% 正文）应能显著缩小 BERT 与词袋的差距，这需要额外的对照实验验证。
4. 词向量的聚合方式较为简单。本实验按要求使用算术平均。可尝试 TF-IDF 加权平均、SIF 加权（smooth inverse frequency）等更优的聚合方式。
5. 未做多次运行的方差统计。所有实验固定 seed=42 运行一次。Task 1/2 是完全确定性的；Task 3 涉及 GPU + cuDNN，个别算子存在非确定性，重复运行指标可能有小数点后第 3~4 位的波动。

附录 A：代码结构与复现步骤

A.1 代码结构

```
HW-1/
├── common/
│   ├── data.py               # 数据加载 + 固定 80/10/10 划分（落盘缓存）
│   ├── text.py               # nltk 分词 + 磁盘缓存
│   ├── embeddings.py         # GloVe 解析 / Word2Vec 训练 / 文档向量平均
│   └── modeling.py           # Logistic Regression 训练与评价（三 Task 共用口径）
├── scripts/
│   ├── check_env.py          # 依赖自检
│   ├── inspect_data.py       # 数据集结构勘察
│   ├── analyze_lengths.py    # 文档长度分布 / max_len 截断程度分析
│   └── compare_results.py    # 汇总三个 Task 生成对比表
├── task1_bow.py              # Task 1
├── task2_word2vec.py         # Task 2
├── task3_bert.py             # Task 3
├── splits/                   # 固定划分索引
├── results/                  # 指标 JSON 与对比表
├── requirements.txt
└── README.md                 # 详细运行说明
```

A.2 复现步骤

```
# 1. 环境准备
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pip install torch --index-url https://download.pytorch.org/whl/cu124
.\.venv\Scripts\python.exe -c "import nltk; nltk.download('punkt'); nltk.download('punkt_tab')"

# 2. 数据准备：将课程提供的 nyt.csv / ag.csv 放入 datasets/ 目录
.\.venv\Scripts\python.exe scripts\inspect_data.py

# 3. 依次运行三个 Task
.\.venv\Scripts\python.exe task1_bow.py          # 约 1 分钟（含首次分词）
.\.venv\Scripts\python.exe task2_word2vec.py     # 约 4 分钟（含首次解压 GloVe）
.\.venv\Scripts\python.exe task3_bert.py         # GPU 约 2 分钟

# 4. 生成横向对比表
.\.venv\Scripts\python.exe scripts\compare_results.py
```

A.3 结果文件说明

文件	内容
`results/task1_results.json`	两种词袋表示的 val/test 指标、分类别 P/R/F1、混淆矩阵
`results/task2_results.json`	四组词向量的指标、词表大小、token 覆盖率、Word2Vec 超参数
`results/task3_results.json`	BERT 各轮验证曲线、截断统计、分类别指标、混淆矩阵
`results/comparison.md`	三组方法汇总对比表